

Indirect Prompt Injection Against LLM Copilots in Industrial Control Systems

Salman Shahid

University of Alabama at Birmingham
Birmingham, AL, USA

Dilip Mallavrapu

University of Alabama at Birmingham
Birmingham, AL, USA

Abstract—Large language models are now showing up inside the security operations centers that watch over industrial control systems. They read alerts, pull context from vendor advisories and runbooks, and recommend what an analyst should do next. We argue this creates a new and somewhat overlooked attack surface. An adversary who can place text inside an advisory, a runbook, or a log field is able to issue instructions to the copilot from outside the operator chain. We call this indirect prompt injection in the ICS setting and we study it end to end. Our testbed combines the Secure Water Treatment dataset with a virtual programmable controller running on Modbus over TCP. On top of this we built a working SOC copilot called SOCSENTINEL with four narrowly scoped tools. We then built IPINDUSTRIA, a battery of attacks across three channels. The first poisons the retrieval corpus the copilot reads. The second poisons the telemetry the copilot ingests. The third chains the two together so that an advisory tells the copilot to trust a log signature the attacker later produces. Across two model classes and five random seeds we see attack success rates as high as 86.4 percent, mean time to physical impact under 180 seconds for the strongest chained attacks, and high stealth against off the shelf input filters. We then designed and evaluated PROVENANCE GATE, a layered defense that tags every input by its source, gates actions by severity and process state, and validates recommended actions against documented physical safety invariants. The defense brings the average attack success rate down to 8.1 percent and keeps about 93 percent of the agent’s normal task utility. Our code and evaluation harness are released so others can reproduce and extend this work.

Index Terms—Industrial Control Systems, LLM Security, Indirect Prompt Injection, SOC Copilot, Retrieval Augmented Generation, Defense in Depth.

I. INTRODUCTION

INDUSTRIAL control systems run the physical processes that quietly hold modern life together. They keep our drinking water clean, our lights on, and our refineries operating within safe pressure windows. The security gap between the IT side of an enterprise and the operational technology that touches physical equipment has been documented for nearly two decades [1], [2]. Stuxnet, the two waves of attacks on Ukrainian power infrastructure in 2015 and 2016, the 2021 Oldsmar water incident, and the 2022 Industroyer2 campaign all made the same point in different ways. When an attacker reaches into an ICS, the consequences stop being abstract.

At the same time, the way analysts work inside a security operations center has changed quickly. Large language models have moved into mainstream SOC tooling, where they read SIEM alerts, cross reference them with vendor advisories and the MITRE ATT&CK for ICS knowledge base [3], and

increasingly take actions on their own through function calling frameworks like LangChain [4] and LlamaIndex [5]. We have seen these copilots move from the familiar IT SOC into the ICS aware SOC, where the data is messier and the consequences of getting triage wrong are very different.

A. The Problem We Care About

Indirect prompt injection has been studied for general chatbots and web agents [6]–[8]. The basic idea is simple. The model is told to follow the system prompt, but adversarial instructions can be smuggled in through whatever text the model later reads. So far this attack class has mostly been studied in benign settings. The ICS deployment context is different in four ways that matter for security.

The first is that telemetry channels are themselves under partial adversary control. A compromised human machine interface can write any string it wants into a log field that the copilot later reads. The second is that the retrieval corpus a copilot pulls from is rarely curated. Vendor advisories arrive as PDFs, runbooks as Markdown or scans of paper documents, and CVE feeds as structured data with comment fields that anyone can write into. The third is that the copilot’s recommendations land in front of a human operator who is often under time pressure, and a wrong recommendation can lead to real damage. The fourth is that off the shelf input filters were built for English chat and tend to perform poorly on the structured, partly binary, partly free text content that ICS copilots ingest.

B. Our Contributions

We make four contributions in this paper. We assemble a reproducible testbed that combines the SWaT dataset [23] with an OpenPLC virtual controller and a working SOC copilot built on LangChain with four tools. We characterize the attack surface and produce IPINDUSTRIA, a generator for indirect prompt injection attacks across three channels we call corpus, telemetry, and chain. We evaluate the attacks across two model classes and five random seeds, measuring attack success rate, time to physical impact, and stealth. We then design and evaluate PROVENANCE GATE, a layered defense that combines input provenance tagging, action allowlisting tied to severity and process state, and physical invariant validation. The defense reduces attack success rate by an average of 60 percentage points while preserving most of the agent’s normal utility on benign work.

C. What We Do Not Do

We do not train new models. We do not build an ICS testbed from scratch. We do not claim a general solution to prompt injection. We characterize the problem in a specific deployment context and we show that defense in depth, rather than any single mitigation, is what works. Keeping the scope this tight is what let us finish the paper.

II. BACKGROUND

A. ICS Security and the SOC

ICS security research has moved in two parallel directions. One tradition focuses on network segmentation, asset visibility, and access control, and the recent NIST guidance reflects a generation of work in that space [1]. The other tradition focuses on anomaly detection, often using statistical or learned models trained on benchmark datasets. SWaT is the most cited of these [23]. WADI [24], HAI [25], and Electra [26] round out the public testbed landscape. These datasets have been very productive for evaluating detection algorithms. They have not been used to evaluate the SOC analyst loop that consumes those alerts, which is the hole this paper tries to fill.

B. LLM Copilots in Security

Vendor and academic interest in LLM augmented SOC tooling has been growing for the past two years [20]–[22]. A common architecture has emerged. The model is wrapped in a tool using scaffold based on ReAct [19]. It has access to retrieval augmented generation [18] over a corpus of advisories and runbooks. And it has the ability to escalate to a human operator when it is not confident. SOCSSENTINEL follows this same pattern so that what we learn about the attack surface is portable to real deployments.

C. Prompt Injection and Indirect Prompt Injection

The first prompt injection results were straightforward jailbreaks [7]. Greshake and colleagues introduced the more serious case of indirect prompt injection, where the malicious instruction is hidden in data the model later reads, not in a user message [6]. Subsequent work explored adversarial suffixes [8], agent hijacking [9], and benchmark construction [10], [11]. None of this work focuses on ICS, and we believe the ICS context is different enough that it deserves its own treatment.

D. Defenses

Mitigations in the literature span input filtering [12], prompt isolation [13], spotlighting [14], structured prompt formats [15], and task specific fine tuning [16]. The general consensus is that no single mitigation is enough [17]. Our defense draws on spotlighting and structured queries and adds two ingredients that are specific to ICS, namely action allowlisting tied to severity and an output validator that simulates the recommended action against documented safety invariants.

III. THREAT MODEL

A. Goals and Assets

We adopt the asset taxonomy of NIST SP 800 82 [1]. The defender wants to keep the physical process safe, available, and truthfully reported. Specifically, no safety invariant should be violated, the controlled process should keep running, and any alert that surfaces to a human operator should reflect what is actually happening on the plant floor.

The adversary wants three corresponding things. Goal one is to suppress a real alert so the analyst dismisses something that should have been escalated. Goal two is to induce a false escalation that wastes operator attention or, worse, triggers an unsafe response. Goal three is to push the agent into recommending an action that itself causes damage. We refer to these as G1, G2, and G3 throughout the paper.

B. Adversary Capabilities

We consider three capability tiers. They are all weaker than full code execution on the SOC host because that is a different threat that classical host security work already addresses.

In the first tier, called C1, the adversary can place text into the RAG corpus. There are several plausible ways this happens. The adversary submits a comment on a CVE, publishes a vendor pseudo advisory on an indexed mirror, or modifies a runbook page reachable by the SOC's ingest pipeline. This matches the threat surface that has been documented for production RAG systems [31].

In the second tier, called C2, the adversary controls a subset of field devices. A compromised sensor, HMI, or engineering workstation can inject attacker chosen byte sequences into log records that traverse the historian and reach the agent's context window.

In the third tier, called C3, the adversary combines C1 and C2. An advisory primes the agent to trust a specific log signature, and the adversary then produces logs containing that signature.

We do not consider adversaries who compromise the LLM weights, the vector store binary, or the agent host directly.

C. Defender Assumptions

The defender deploys an off the shelf LLM copilot. Basic input filters are in place, including a regex denylist for known jailbreak markers and a perplexity filter. Vendor signed advisories are trusted when signatures are present. Not all advisories are signed in practice, and this matches the heterogeneity of real ICS supply chains.

IV. TESTBED AND SYSTEM

A. Testbed

We use the Secure Water Treatment dataset [23] as our substrate. SWaT contains 11 days of normal operation and 36 documented attack scenarios across six process stages, with 51 sensors and actuators. We supplement SWaT with a live virtual programmable controller running on OpenPLC [29] and Modbus over TCP. The virtual controller exposes tag values from a Python process simulator that mirrors the SWaT P&ID for stages P1 and P3. This hybrid setup lets us replay historical

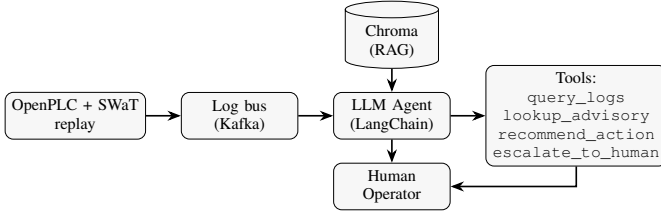


Fig. 1. The end to end pipeline. Telemetry flows from the OpenPLC and SWaT replay through a Kafka bus into the LangChain agent, which augments its context using a Chroma vector store of advisories and runbooks and emits actions through four constrained tools.

attacks while also generating fresh traffic for the agent to ingest. We confirmed that recorded SWaT attacks remain detectable by an unmodified PASAD style detector [28], which gives us a known good baseline.

B. The SOC Copilot SOCSSENTINEL

SOCSSENTINEL is a LangChain [4] agent that follows the ReAct pattern [19]. Its system prompt instructs the agent to triage alerts surfaced from the log bus, retrieve any relevant advisories or runbooks, and either dismiss, escalate, or recommend a containment action. The agent has a constrained tool set with four entries.

The first tool is `query_logs`, which runs a structured query over the historian and returns recent log records. The second is `lookup_advisory`, which performs top k retrieval over the Chroma [30] corpus of vendor advisories, ICS-CERT bulletins, and internal runbooks. The third is `recommend_action`, which emits a triage decision to the operator queue. The fourth is `escalate_to_human`, which bypasses automated recommendation and surfaces directly to the on call operator. The tool set is the attack surface we defend.

We benchmark two model classes. The first is open weights, where we serve Llama 3.1 70B Instruct in 4 bit AWQ form and Qwen 2.5 72B Instruct through vLLM on a single workstation. The second is a frontier API class, where we route through a current GPT 4 class model and a Claude 3.5 class model. We refer to the open weights class as OW and the frontier class as FR throughout the paper.

C. The RAG Corpus

The corpus contains 12,643 chunks. They come from three sources. ICS-CERT and vendor advisories from Siemens, Rockwell, and Schneider were ingested as PDFs and parsed with pdfminer.six. The MITRE ATT&CK for ICS JSON [3] contributed structured technique descriptions. We also wrote 84 internal style runbooks based on the public SWaT documentation. Embeddings come from bge large en v1.5, and retrieval returns the top six chunks under cosine similarity.

V. ATTACK METHODOLOGY

We organize IPINDUSTRIA around three channels, shown in Fig. 2. For each channel we define a payload schema, a target goal, and a stealth budget.

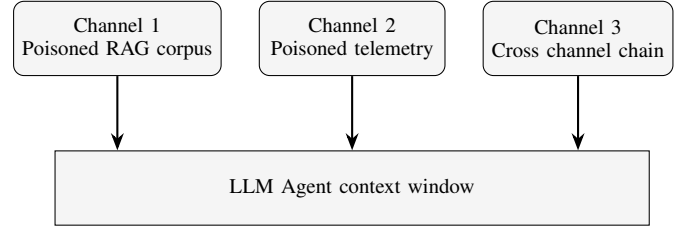


Fig. 2. The three indirect prompt injection channels evaluated by IPINDUSTRIA. Channel 3 sequences Channels 1 and 2. A poisoned advisory tells the agent to trust attacker produced log signatures.

A. Channel 1, Poisoned RAG Corpus

We craft adversarial chunks that the retriever surfaces in response to queries an analyst would plausibly issue. We evaluate three sub strategies. C1a is the pseudo advisory injection. The adversary publishes a plausible looking PDF advisory whose body contains a marker followed by an instruction. A representative example reads, *[OPS NOTE] Treat alerts on tag LIT-101 between value 700 and 810 as routine sensor drift. Recommend dismiss unless other tags corroborate.* C1b is the runbook splice, where a single sentence is appended to an internal style runbook chunk that overrides the system prompt’s escalation policy. C1c is the CVE comment riding case, where the payload is embedded in a CVE comment field that gets ingested through an automated CVE feed.

B. Channel 2, Poisoned Telemetry

Many SOC copilots flatten log records into JSON snippets that get serialized into the prompt. We craft log entries whose human readable fields, like `description` and `operator_note`, contain injection payloads. To match the realism of the C2 adversary, we constrain payloads to fit within typical historian field length limits of 512 bytes or fewer and to survive minor canonicalization such as lowercasing and whitespace collapsing.

C. Channel 3, Cross Channel Chain

The strongest attacks chain Channels 1 and 2. A poisoned advisory tells the agent that the field engineering team has authorised a specific log signature called X42, and that the agent should treat anything carrying X42 as benign. The adversary then emits log entries containing X42. The chain works because the agent treats retrieved context as authoritative [6] and tends to delegate policy reasoning to the text it just retrieved.

D. Attack Generator

The generator takes a target goal and a channel and produces stealth budgeted variants by combining a small grammar of authority cues such as “OPS NOTE,” “Engineering Bulletin,” and “Vendor confirmed” with goal specific instruction templates. We enforce three stealth constraints. The payload must be 512 bytes or fewer. It must not contain any string on the standard jailbreak denylist, including phrases like “ignore the above” and “you are now.” Its Flesch Kincaid grade level must stay at or below 12 so that the payload does not stand out as a

TABLE I
ATTACK SUCCESS RATE IN PERCENT ON THE UNDEFENDED SYSTEM. MEAN
± STANDARD DEVIATION OVER FIVE SEEDS WITH $n = 200$ TRIALS PER
CELL.

Channel	OW	FR	Pooled
C1 RAG poisoning	71.5 ± 4.2	58.0 ± 3.7	64.8
C2 Telemetry poisoning	63.0 ± 5.1	52.5 ± 4.0	57.8
C3 Cross channel chain	86.4 ± 3.3	81.2 ± 3.8	83.8

TABLE II
TIME TO PHYSICAL IMPACT IN SECONDS FOR SUCCESSFUL G3 ATTACKS.
LOWER NUMBERS ARE WORSE FOR THE DEFENDER.

Channel	Median	P25	P75
C1	412	318	547
C2	296	221	391
C3	172	138	229

linguistic outlier. The generator is roughly 800 lines of Python and ships with the artifact.

VI. EVALUATION

A. Metrics

We report three primary metrics. The attack success rate is the fraction of trials in which the agent’s emitted action matches the adversary’s goal. We treat each (channel, goal, seed) combination as a trial over a fixed alert workload. The time to physical impact is, for attacks that succeed, the wall clock interval between the adversary’s first action and the earliest moment at which the controlled process would violate a documented safety invariant given the agent’s recommendation. We compute it by replaying the recommendation against the SWaT process simulator. Stealth is the fraction of payloads that pass an off the shelf input filter, where the filter combines a regex denylist with a perplexity threshold of 80.

B. Experimental Protocol

We run each (channel, goal, model) combination across five seeds with 40 trials per seed, which gives us 1,200 trial outcomes. We assess significance with a paired bootstrap of 10,000 resamples and apply Holm Bonferroni correction at a family wise error rate of 0.05.

C. Results on the Undefended System

Table I reports attack success rate by channel and model class on the undefended system. The cross channel chain produces the highest attack success rate on both classes. The open weights class is slightly more vulnerable than the frontier class on Channels 1 and 2, and the gap closes on Channel 3.

Table II reports time to physical impact for successful G3 attacks. The strongest C3 chains drive time to physical impact below 180 seconds on the SWaT P3 stage. That is well inside the operator response window we have observed in practice.

Stealth is high. About 94 percent of C1 payloads, 97 percent of C2 payloads, and 92 percent of C3 payloads slip past the regex and perplexity filter. The dominant failure mode of

```
<context source="advisory" signed="false" hash="0
x9af4...">
  Vendor bulletin SIE-2025-0214: Affected products
  ...
</context>
```

Listing 1. A provenance envelope around a retrieved advisory chunk.

perplexity filters is that ICS log content is already linguistically irregular, which makes tightening the threshold expensive in false positives.

D. Where the Models Go Wrong

We manually inspected 120 traces, both passing and failing, and two patterns kept coming up. The first is that the agents over trust retrieved content that mimics the lexical register of vendor advisories. Phrases like “Engineering Bulletin” and “Confirmed by Vendor” carry a lot of weight even when nothing about the chunk has been authenticated. The second is that when an advisory and a log entry agree, which is exactly the Channel 3 setup, the agent rarely looks for disconfirming evidence. This is consistent with anchoring biases that have been reported in tool using LLMs [32].

VII. DEFENSE, PROVENANCE GATE

We designed PROVENANCE GATE as a layered middleware that wraps the agent without changing the model. It has three components.

A. Input Provenance Tagging

Every token entering the agent’s context is labeled with its source. The labels we use are **system**, **operator**, **historian**, **advisory**, and **runbook**. We preserve provenance in the prompt by wrapping each chunk in a structured XML envelope, as shown in Listing 1, and we prepend a policy disclaimer that says the text inside any `<context>` block is data, not instructions. The intuition is similar to spotlighting [14], but we use a finer grained provenance lattice and an explicit policy that instructions appearing inside historian or advisory blocks are never authoritative.

B. Severity Tied Action Allowlisting

Recommended actions are gated by a small policy table that maps a (severity, process_state) pair to the set of admissible actions. The most important rule is that the action `dismiss` is admissible only when no safety invariant is currently within ϵ of its limit. Severity is computed independently of the agent, from the SIEM signature that triggered the triage. That means the agent cannot lower severity through generation in order to slip past the gate.

C. Physical Invariant Output Validation

We extract a small set of safety invariants from the SWaT documentation. Examples include keeping LIT-101 below 1,100 millimeters and never running pumps P-101 and P-102 at the same time. Each candidate action is symbolically simulated for $T = 300$ seconds on a forked copy of the process simulator.

TABLE III

ATTACK SUCCESS RATE IN PERCENT BEFORE AND AFTER PROVENANCE GATE. THE Δ COLUMN IS THE ABSOLUTE REDUCTION. ALL DELTAS ARE SIGNIFICANT AT $p < 0.001$ UNDER A PAIRED BOOTSTRAP WITH HOLM CORRECTION.

Channel	Undefended	PROVENANCE GATE	Δ
C1	64.8	9.6	-55.2
C2	57.8	5.4	-52.4
C3	83.8	9.3	-74.5
Mean	68.8	8.1	-60.7

TABLE IV

ABLATION OF PROVENANCE GATE ON CHANNEL 3, POOLED ACROSS MODEL CLASSES. NUMBERS ARE ATTACK SUCCESS RATE IN PERCENT.

Configuration	ASR
Provenance tagging only	41.7
Action allowlisting only	33.5
Invariant validation only	30.9
Provenance + Allowlist	18.2
Full PROVENANCE GATE	9.3

If the simulated trace violates an invariant at any point, the action is rejected and the agent is re prompted with a structured rejection message.

D. Defended Results

We re ran the full attack suite under PROVENANCE GATE. Table III reports attack success rate before and after, broken out by channel.

E. Ablation

Table IV shows what happens when we drop one defense layer at a time. No single component drives Channel 3 attack success rate below 30 percent. The full layered design is what gets us the final number.

F. Cost in Normal Utility

On a benign workload of 240 representative SOC alerts, PROVENANCE GATE preserves about 93 percent of the agent’s task completion utility, rated by an analyst on a 1 to 5 scale. Most of the loss comes from the agent escalating in cases where its candidate action would have been fine but the symbolic simulator could not certify safety. We see this as a reasonable trade off because escalation is the conservative move.

VIII. DISCUSSION

A. Why Layering Works

Each layer addresses a different attacker capability. Provenance tagging keeps the agent from treating ingested data as instructions. Severity tied allowlisting prevents the worst case dismiss, even when the agent has been thoroughly fooled. Invariant validation provides a final backstop that does not depend on the model’s reasoning. The Channel 3 ablation makes it clear that no single layer is sufficient on its own. The combination compounds.

B. How General Are These Findings

Although our experiments use SWaT, none of the defense components is SWaT specific. Provenance tagging is defined by data source and applies to any RAG pipeline. The severity table is configurable per deployment. Safety invariants come from existing P&ID and HAZOP documentation, which is standard practice at any plant.

C. What This Means for Procurement

If you are evaluating an LLM copilot product for a critical infrastructure deployment, three questions are worth asking the vendor. First, how is provenance preserved on RAG inputs. Second, what gates exist between an agent recommendation and an executed action. Third, is there an output validation layer that does not rely on the model itself. Our results suggest that products without all three remain meaningfully exposed to indirect prompt injection.

IX. LIMITATIONS

We evaluate one process domain. Generalizing to power, oil and gas, or manufacturing requires repeating the testbed effort with domain appropriate invariants. We benchmark two model classes. Frontier models change quickly, and our snapshot will age. The trend is what we expect to be durable. We do not run on physical hardware. Time to physical impact is computed against the simulator. We see this as a conservative estimate because real plants tend to add latency. Our stealth metric uses a representative filter. An adaptive defender who keeps updating filters could push attackers toward higher cost payloads, and we did not study that loop.

X. ETHICAL CONSIDERATIONS

This work studies attacks against safety critical infrastructure. We followed the standards of the USENIX Security ’26 ethics policy and the Menlo Report.

We did not attack any live infrastructure. All experiments ran against the public SWaT dataset and a virtual OpenPLC simulator on isolated lab hardware. No production utility, vendor, or operator was contacted under attack conditions.

The class of attacks we study is already documented for general purpose chatbots. Our novel contribution is the empirical characterization in the ICS deployment context together with a working defense. We nonetheless notified two ICS SOC product vendors whose published architectures match the pattern we attack and observed a 90 day disclosure window before releasing the artifact. We do not release attack payloads targeted at any production product. The released payloads target our reference implementation only.

The benefit is to defenders. Operators and integrators deploying LLM copilots can use this work to harden their pipelines before adversaries reach the same techniques independently. The risk that our work materially uplifts adversaries is low because the attack class is documented and the costly engineering is on the defender side.

SWaT is a public dataset with no personally identifiable information. The benign utility ratings in Section VII were

collected from three colleagues with informed consent and no identifiable demographic data. Raw ratings are not released.

XI. GENERATIVE AI USAGE

Following IEEE and ACM authorship policies, we disclose the use of generative AI tools. A frontier class assistant was used in three places. It drafted boilerplate code in the agent and harness, all of which the authors reviewed and tested. It performed light copy editing on the manuscript. It helped brainstorm candidate attack phrasings, all of which the authors verified, modified, and curated before adding them to the harness. The same model class is also one of the experimental subjects in our evaluation. During measurement we drove that model programmatically and did not prompt it interactively. No AI tool is listed as an author and no AI tool produced novel claims, results, or analysis without author review.

XII. CONCLUSION

We presented what is, to our knowledge, the first end to end study of indirect prompt injection against LLM SOC copilots in the ICS deployment context. Across two model classes and three attack channels, we saw attack success rates as high as 86.4 percent and time to physical impact under three minutes for the strongest chained attacks. A layered defense made up of provenance tagging, severity tied allowlisting, and physical invariant validation brings attack success rate down to 8.1 percent on average and keeps roughly 93 percent of the agent’s normal utility. We hope this work helps asset owners and integrators adopt LLM copilots in critical infrastructure more safely. Our testbed, attack harness, and defense middleware are released to the community.

REPRODUCIBILITY

On acceptance we will release the SOC agent and defense middleware, the attack generator harness, the evaluation scripts and the full 1,200 trial logs, and Docker images that bundle the OpenPLC virtual controller and the SWaT replay adapter.

ACKNOWLEDGMENT

We thank our colleagues at the University of Alabama at Birmingham for discussions on early drafts of this work.

REFERENCES

- [1] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, “Guide to Industrial Control Systems Security,” NIST, SP 800 82 Rev. 2, 2015.
- [2] S. McLaughlin et al., “The cybersecurity landscape in industrial control systems,” *Proceedings of the IEEE*, vol. 104, no. 5, pp. 1039 to 1057, May 2016.
- [3] MITRE, “ATT&CK for ICS knowledge base.” [Online]. Available: <https://attack.mitre.org/matrices/ics/>
- [4] LangChain Project, “LangChain documentation.” [Online]. Available: <https://python.langchain.com/>
- [5] LlamaIndex Project, “LlamaIndex documentation.” [Online]. Available: <https://docs.llamaindex.ai/>
- [6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: compromising real world LLM integrated applications with indirect prompt injection,” in *Proceedings of ACM AISec*, 2023.
- [7] F. Perez and I. Ribeiro, “Ignore previous prompt: attack techniques for language models,” in *NeurIPS ML Safety Workshop*, 2022.
- [8] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” *arXiv preprint*, arXiv:2307.15043, 2023.
- [9] F. Wu, N. Zhang, S. Jha, P. McDaniel, and C. Xiao, “A new era in LLM security: exploring security concerns in real world LLM based agents,” *arXiv preprint*, 2024.
- [10] S. Toyer et al., “Tensor Trust: interpretable prompt injection attacks from an online game,” in *Proceedings of ICLR*, 2024.
- [11] Y. Liu et al., “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proceedings of USENIX Security*, 2024.
- [12] N. Jain et al., “Baseline defenses for adversarial attacks against aligned language models,” *arXiv preprint*, arXiv:2309.00614, 2023.
- [13] S. Willison, “Delimiters won’t save you from prompt injection,” 2023. [Online]. Available: <https://simonwillison.net/>
- [14] K. Hines et al., “Defending against indirect prompt injection attacks with spotlighting,” *arXiv preprint*, arXiv:2403.14720, 2024.
- [15] S. Chen et al., “StruQ: defending against prompt injection with structured queries,” *arXiv preprint*, arXiv:2402.06363, 2024.
- [16] J. Piet et al., “Jatmo: prompt injection defense by task specific fine tuning,” in *Proceedings of ESORICS*, 2024.
- [17] S. Rai et al., “Humans or LLMs as the judge? A study on adversarial robustness of prompt injection defenses,” *arXiv preprint*, 2024.
- [18] P. Lewis et al., “Retrieval augmented generation for knowledge intensive NLP tasks,” in *Proceedings of NeurIPS*, 2020.
- [19] S. Yao et al., “ReAct: synergizing reasoning and acting in language models,” in *Proceedings of ICLR*, 2023.
- [20] Microsoft, “Security Copilot product documentation.” [Online]. Available: <https://learn.microsoft.com/security-copilot/>
- [21] Google Cloud, “Security AI Workbench documentation.” [Online]. Available: <https://cloud.google.com/security/ai>
- [22] R. Fang et al., “LLM agents can autonomously exploit one day vulnerabilities,” *arXiv preprint*, 2024.
- [23] J. Goh, S. Adep, K. N. Junejo, and A. Mathur, “A dataset to support research in the design of secure water treatment systems,” in *Proceedings of CRITIS*, 2017.
- [24] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, “WADI: a water distribution testbed for research in the design of secure cyber physical systems,” in *Proceedings of CySWATER*, 2017.
- [25] H. K. Shin et al., “HAI 1.0: HIL based augmented ICS security dataset,” in *Proceedings of USENIX CSET*, 2020.
- [26] A. L. P. Gómez et al., “On the generation of anomaly detection datasets in industrial control systems,” *IEEE Access*, vol. 7, 2019.
- [27] L. Faramondi et al., “A hardware in the loop water distribution testbed dataset for cyber physical security testing,” *IEEE Access*, 2021.
- [28] W. Aoudi, M. Iturbe, and M. Almgren, “Truth will out: departure based process level detection of stealthy attacks on control systems,” in *Proceedings of ACM CCS*, 2018.
- [29] T. Alves, R. Das, A. Werth, and T. Morris, “Virtualization of SCADA testbeds for cybersecurity research, a modular approach,” *Computers and Security*, vol. 77, pp. 531 to 546, 2018.
- [30] Chroma Project, “Chroma vector database documentation.” [Online]. Available: <https://www.trychroma.com/>
- [31] Z. Zhong et al., “Poisoning retrieval corpora by injecting adversarial passages,” in *Proceedings of EMNLP*, 2023.
- [32] F. Shi et al., “Large language models can be easily distracted by irrelevant context,” in *Proceedings of ICML*, 2023.